

Compose 성능 최적화

1 개요 및 학습목표

개요 및 학습목표 Smart Recomposition

1. 개요 및 학습목표



What is the Smart Recomposition?

개요 및 학습목표

Compose 성능최적화 이론

1.

개요 및 학습목표

이론

Lifecycle
(수명주기)

Rendering Phase
(렌더링 단계)

Stability
(안정성)

Smart Recomposition Optimization

개요 및 학습목표

Compose 성능최적화 실습

1.

개요 및 학습목표

실습

Defer State Reading
(상태 읽기 연기)

Lazy Layout Key
(레이지 레이아웃 키 사용)



Memo App Refactoring

Minimize Calculation
(계산 최소화)

Avoid Backward Writes
(역방향 쓰기 방지)

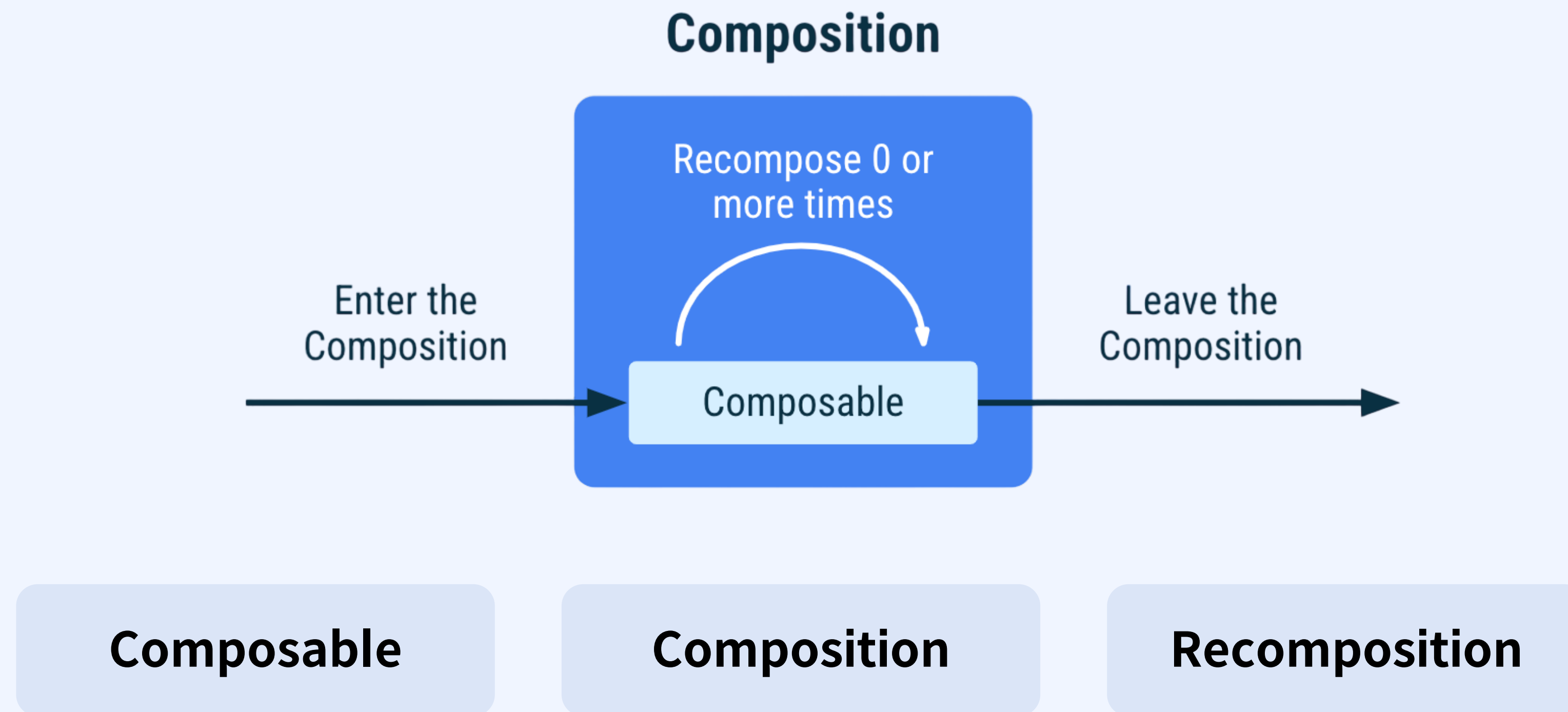
Compose 성능 최적화

2 수명주기 (Lifecycle)

Lifecycle Composition

2.

수명주기
Lifecycle



<https://developer.android.com/jetpack/compose/lifecycle>

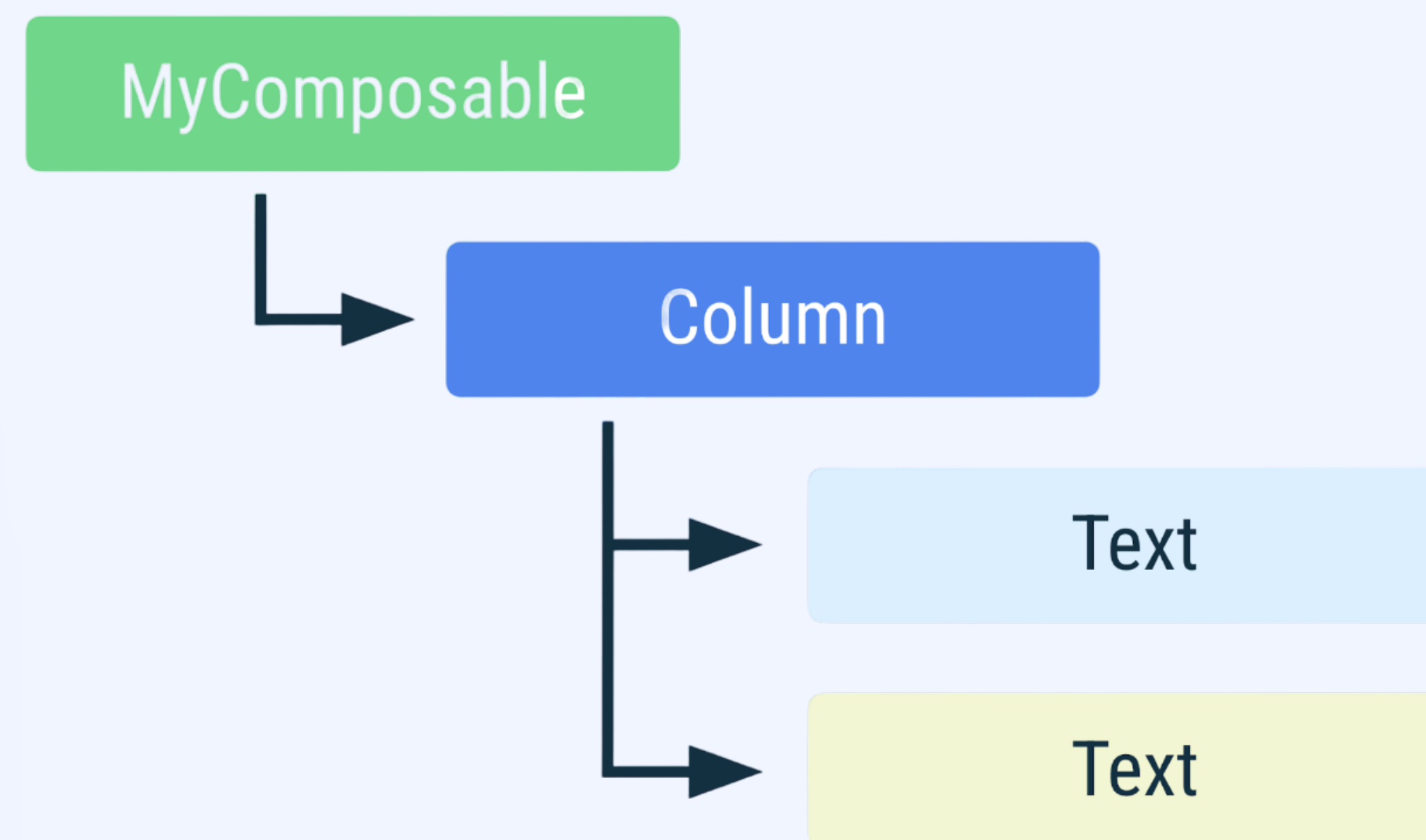
Lifecycle

인스턴스 구분

2.

수명주기
Lifecycle

```
@Composable
fun MyComposable() {
    Column {
        Text("Hello")
        Text("World")
    }
}
```



Lifecycle

콜 사이트를 통한 구분

2.

수명주기
Lifecycle



```
@Composable
fun LoginScreen(showError: Boolean) {
    if (showError) {
        LoginError()
    }
    LoginInput()
}
```

```
@Composable
fun LoginInput() { /* ... */ }
```

LoginScreen

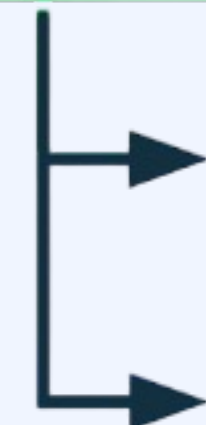


LoginInput

Recomposition
(showError = true)



LoginScreen



LoginError

LoginInput

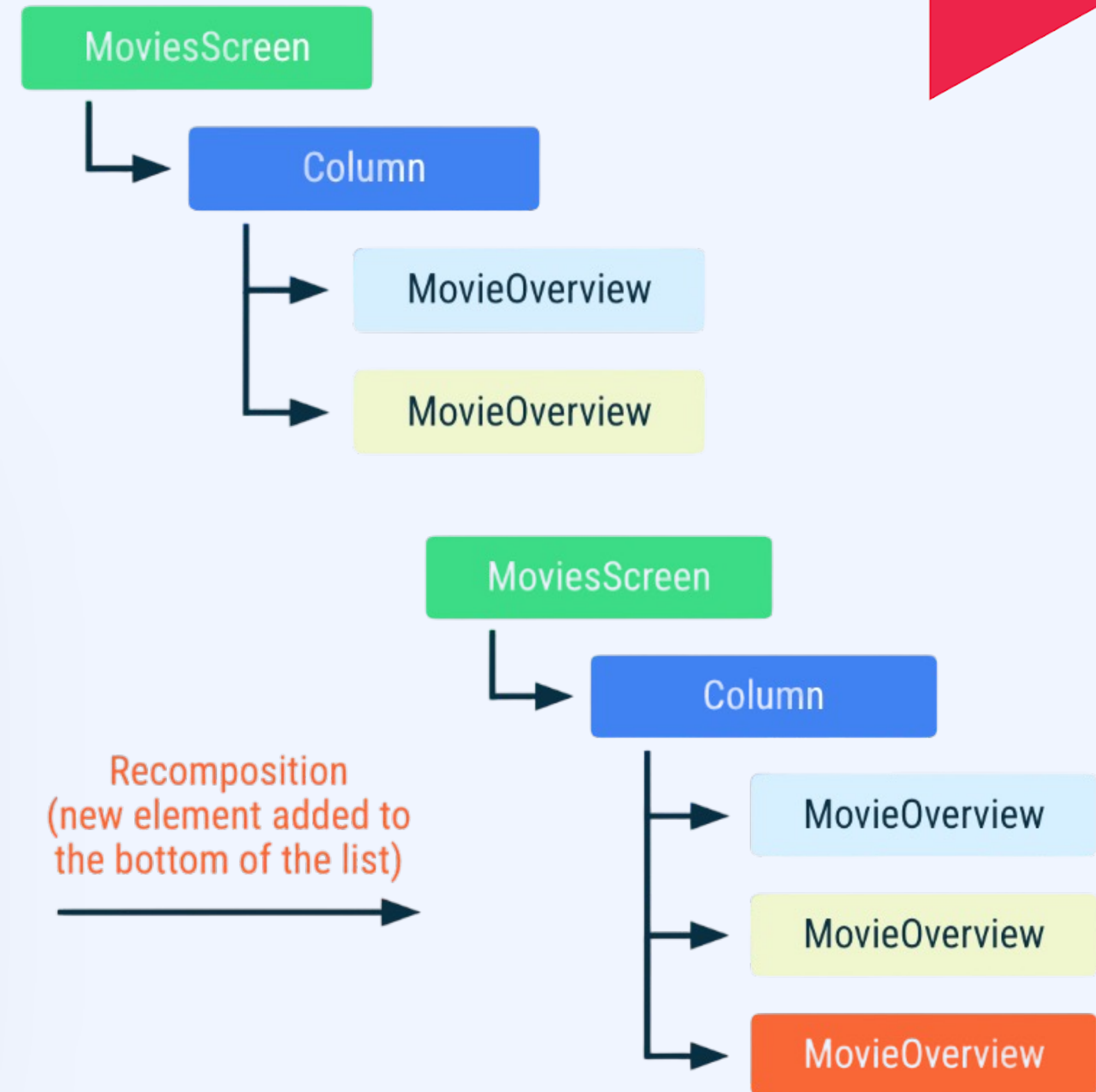
Lifecycle

실행순서를 통한 구분

2.

수명주기
Lifecycle

```
@Composable
fun MoviesScreen(movies: List<Movie>) {
    Column {
        for (movie in movies) {
            MovieOverview(movie)
        }
    }
}
```

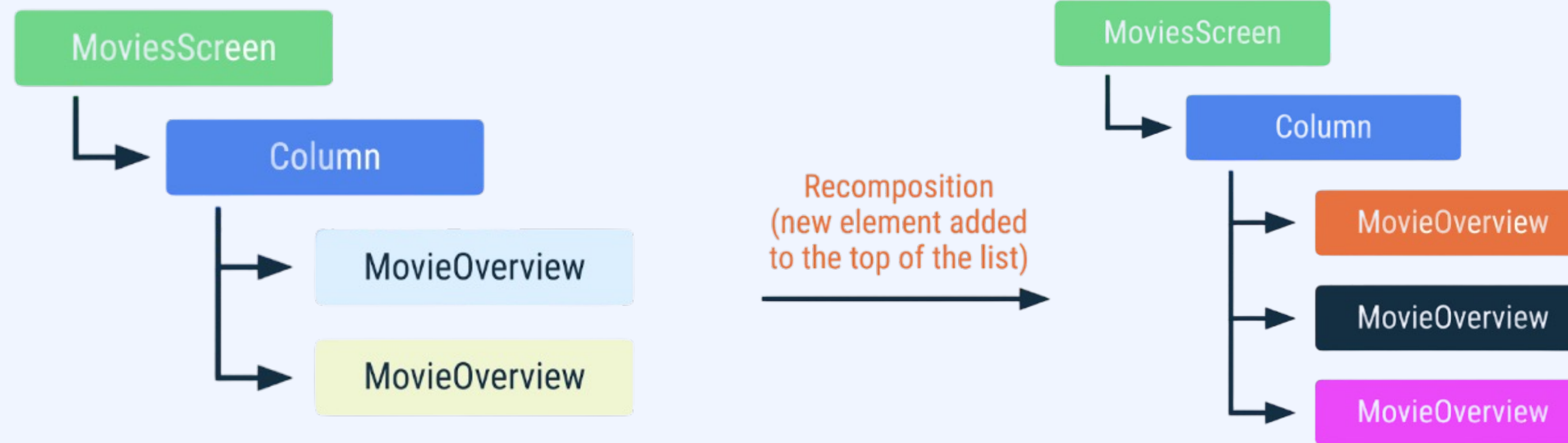


Lifecycle

Lifecycle (수명주기)

2.

수명주기
Lifecycle



```

@Composable
fun MoviesScreen(movies: List<Movie>) {
    Column {
        for (movie in movies) {
            MovieOverview(movie)
        }
    }
}
  
```

```

@Composable
fun MovieOverview(movie: Movie) {
    Column {
        val image = loadNetworkImage(movie.url)
        MovieHeader(image)
    }
}
  
```

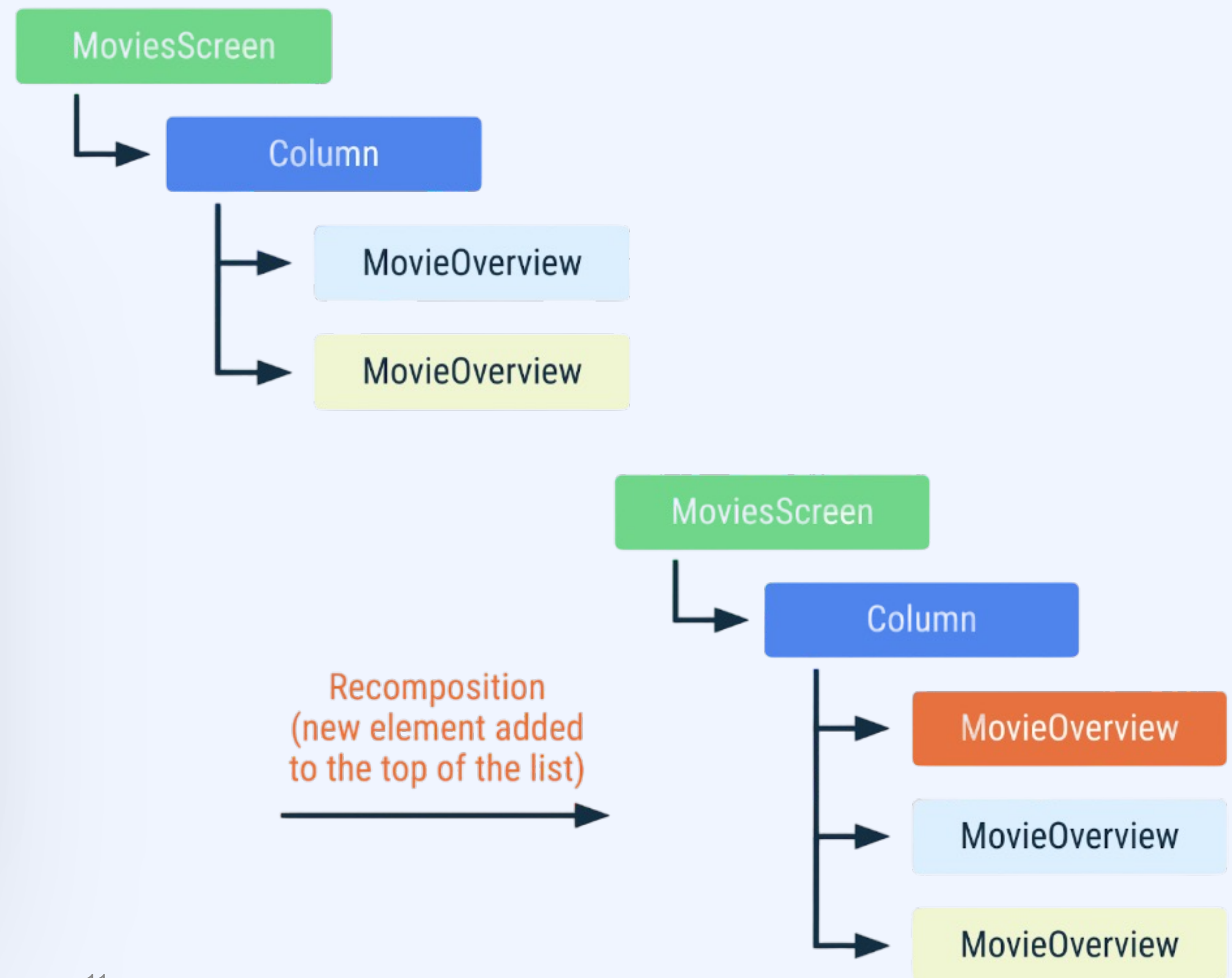
Lifecycle

Key를 사용한 아이템 구분

2.

수명주기
Lifecycle

```
@Composable
fun MoviesScreen(movies: List<Movie>) {
    Column {
        for (movie in movies) {
            key(movie.id) {
                MovieOverview(movie)
            }
        }
    }
}
```



Lifecycle

Lifecycle (수명주기)

2.

수명주기
Lifecycle

```
@Composable
fun MoviesScreen(movies: List<Movie>) {
    Column {
        for (movie in movies) {
            key(movie.id) {
                MovieOverview(movie)
            }
        }
    }
}
```

```
@Composable
fun MoviesScreen(movies: List<Movie>) {
    LazyColumn {
        items(movies, key = { movie -> movie.id }) { movie ->
            MovieOverview(movie)
        }
    }
}
```

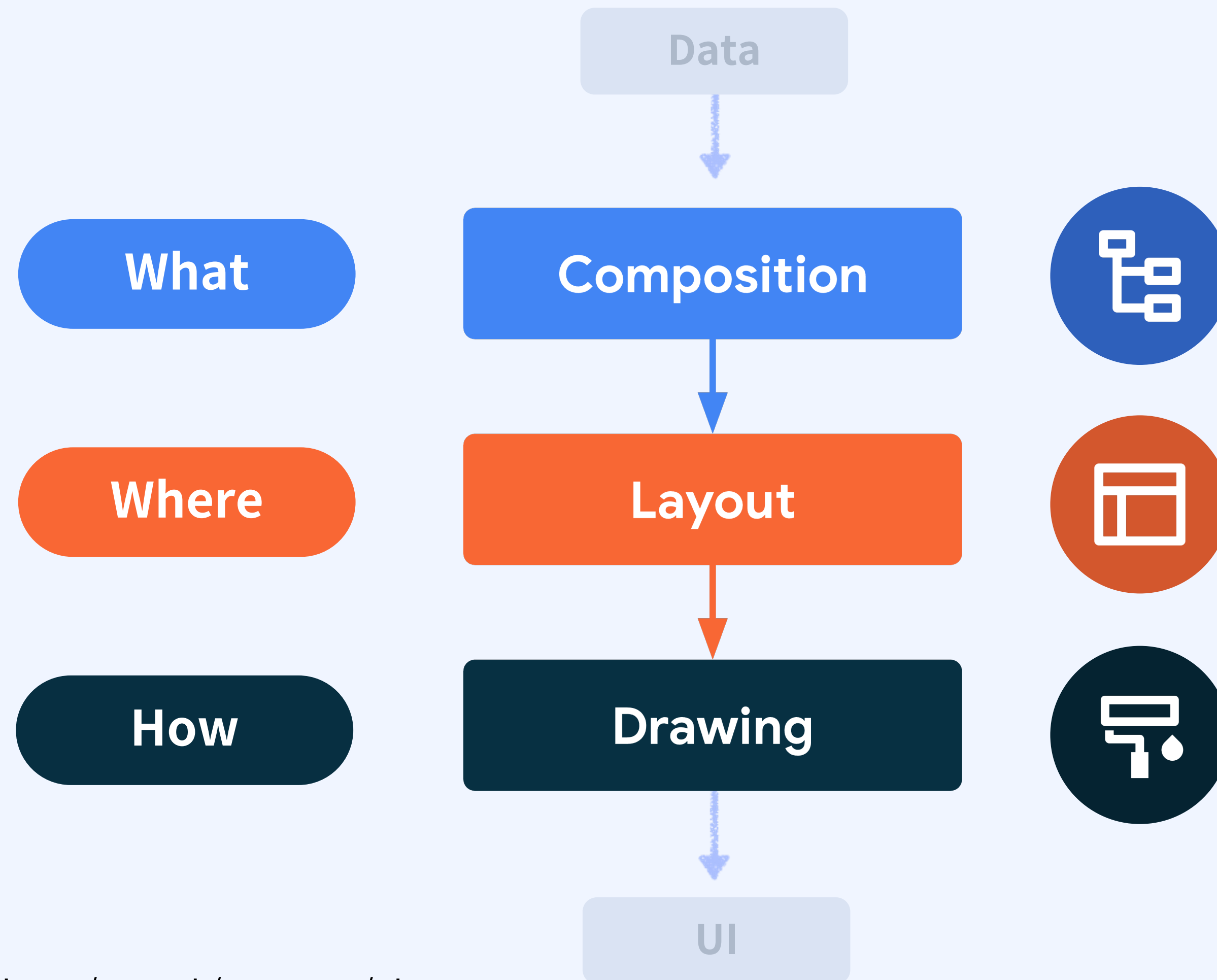
Compose 성능 최적화

3 렌더링 단계 (Rendering Phase)

Rendering Phase

Composition - Layout - Drawing

3.
렌더링 단계
Rendering Phase

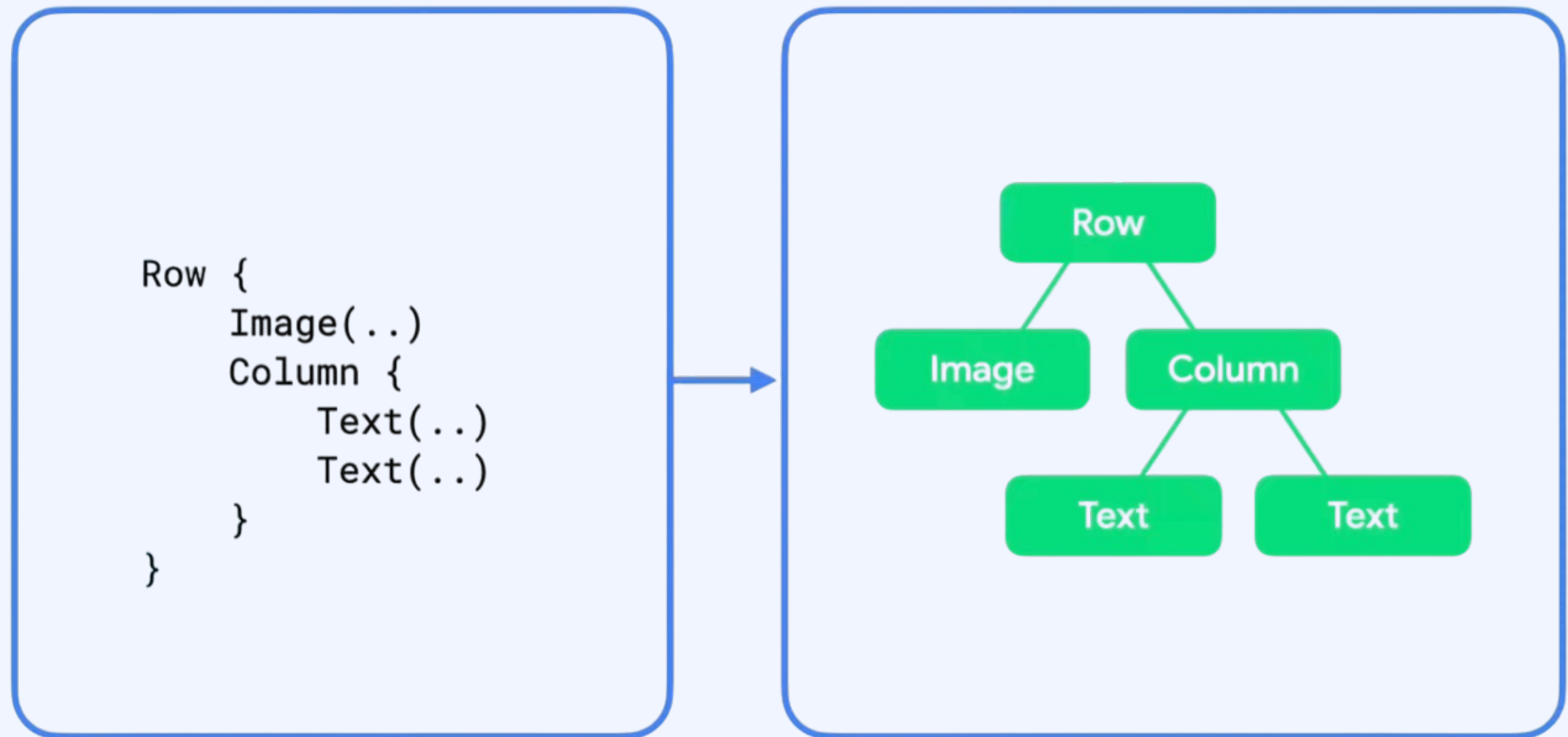
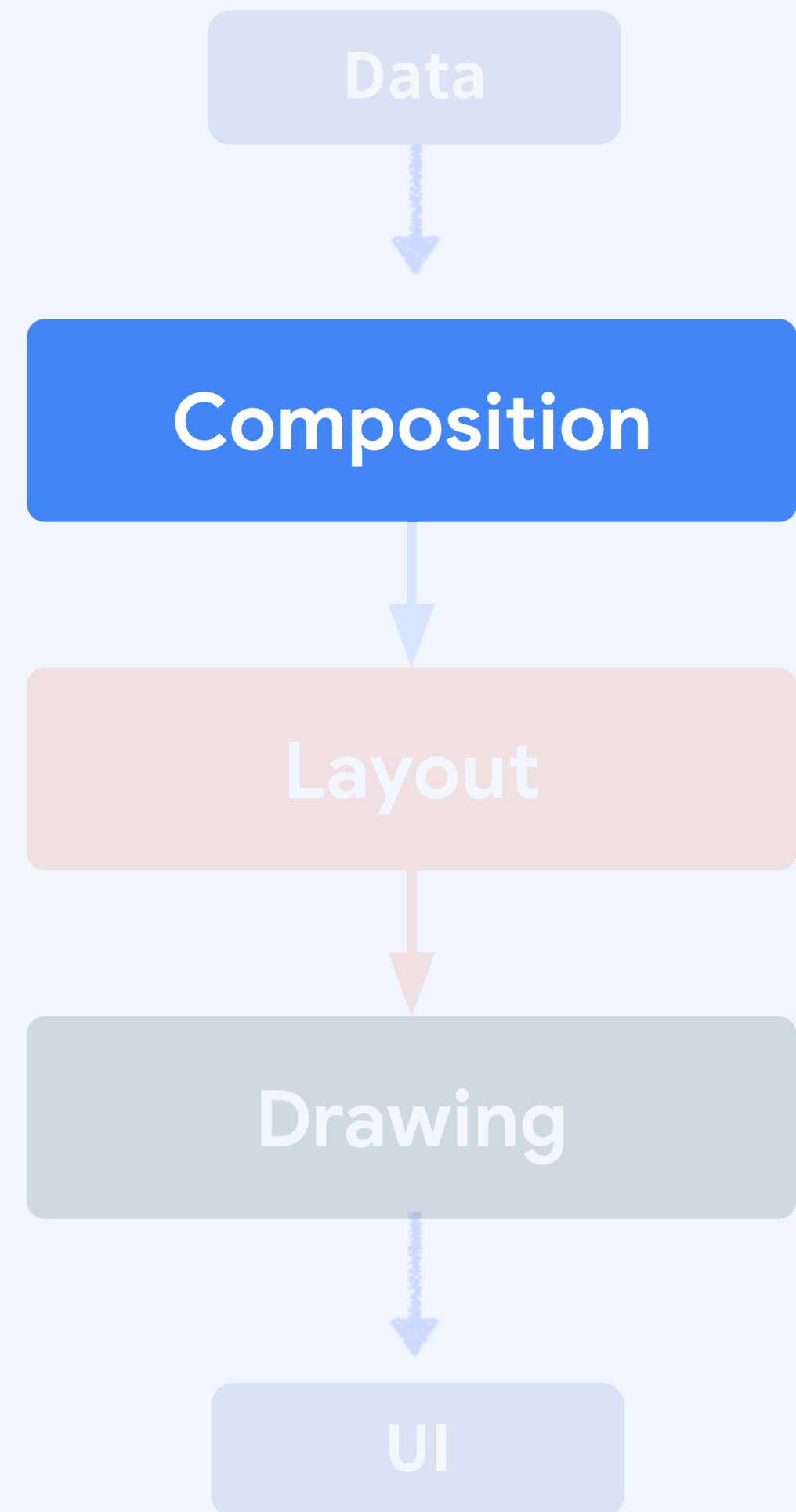


<https://developer.android.com/jetpack/compose/phases>

Rendering Phase Composition

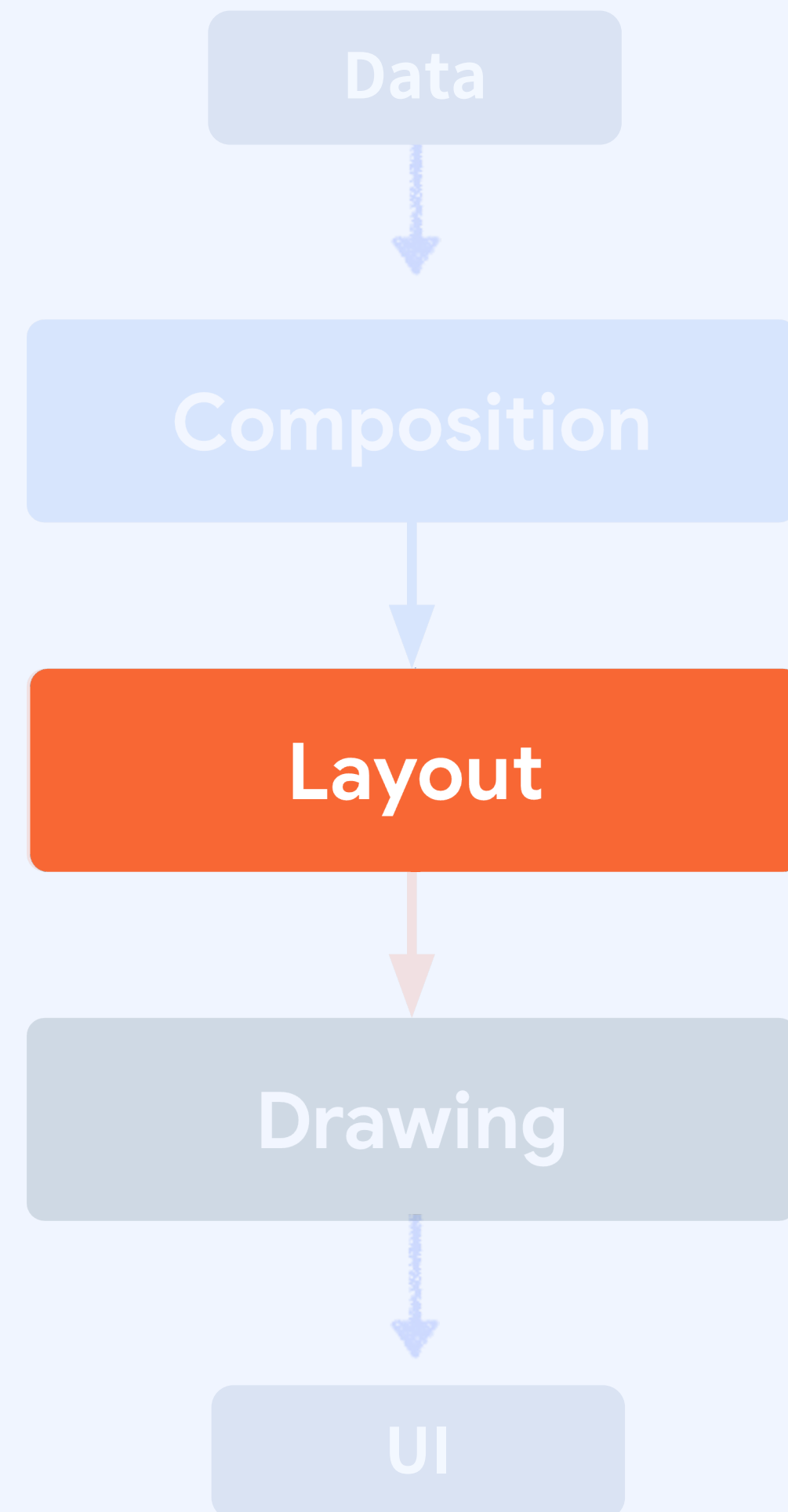
3.

렌더링 단계
Rendering Phase



Rendering Phase Composition

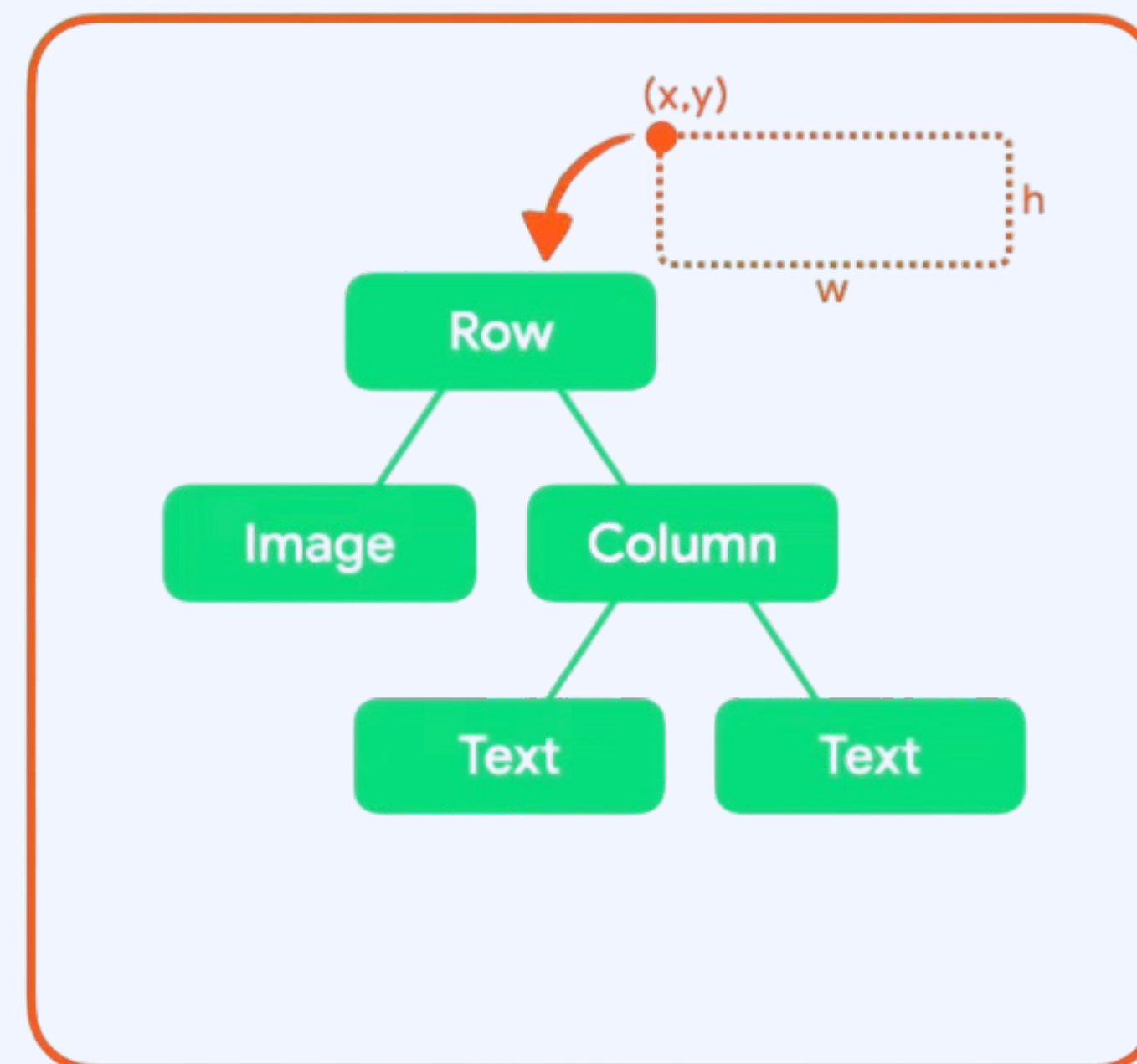
3. 렌더링 단계 Rendering Phase



1. Measure children (자식 측정)

2. Decide own size (본인 크기 측정)

3. Place children (자식 배치)



Rendering Phase Composition

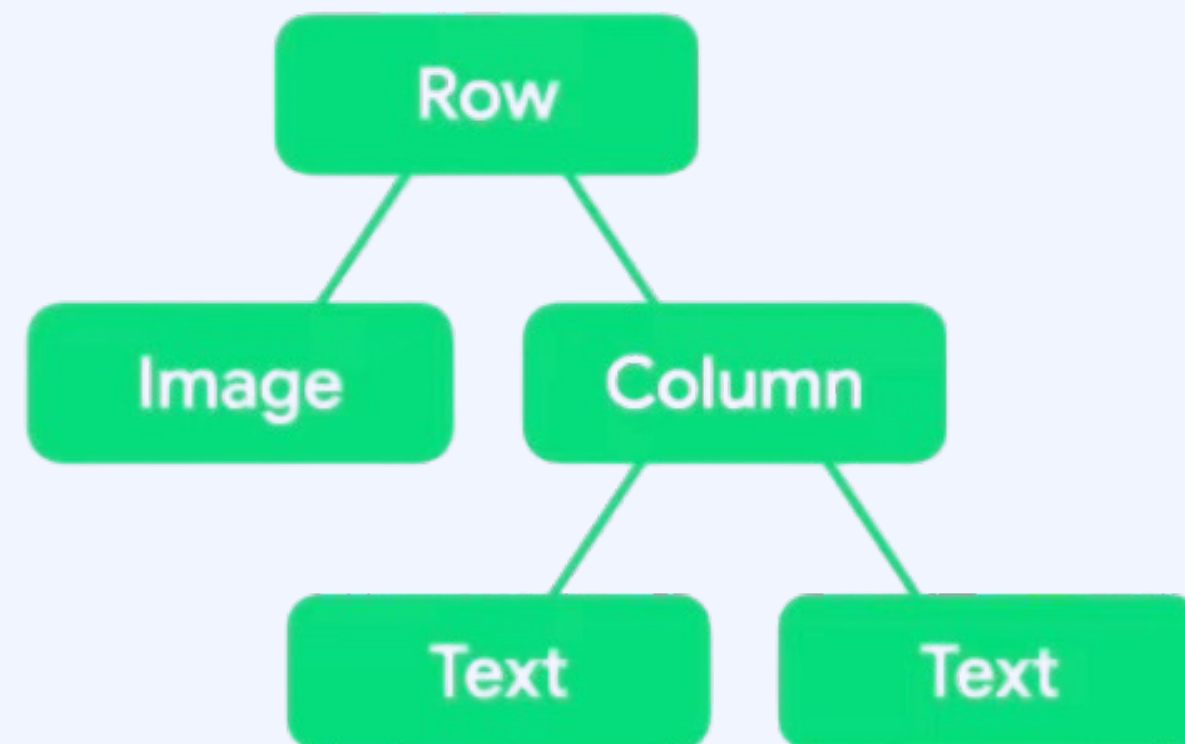
3.

렌더링 단계
Rendering Phase

1. Measure children (자식 측정)

2. Decide own size (본인 크기 측정)

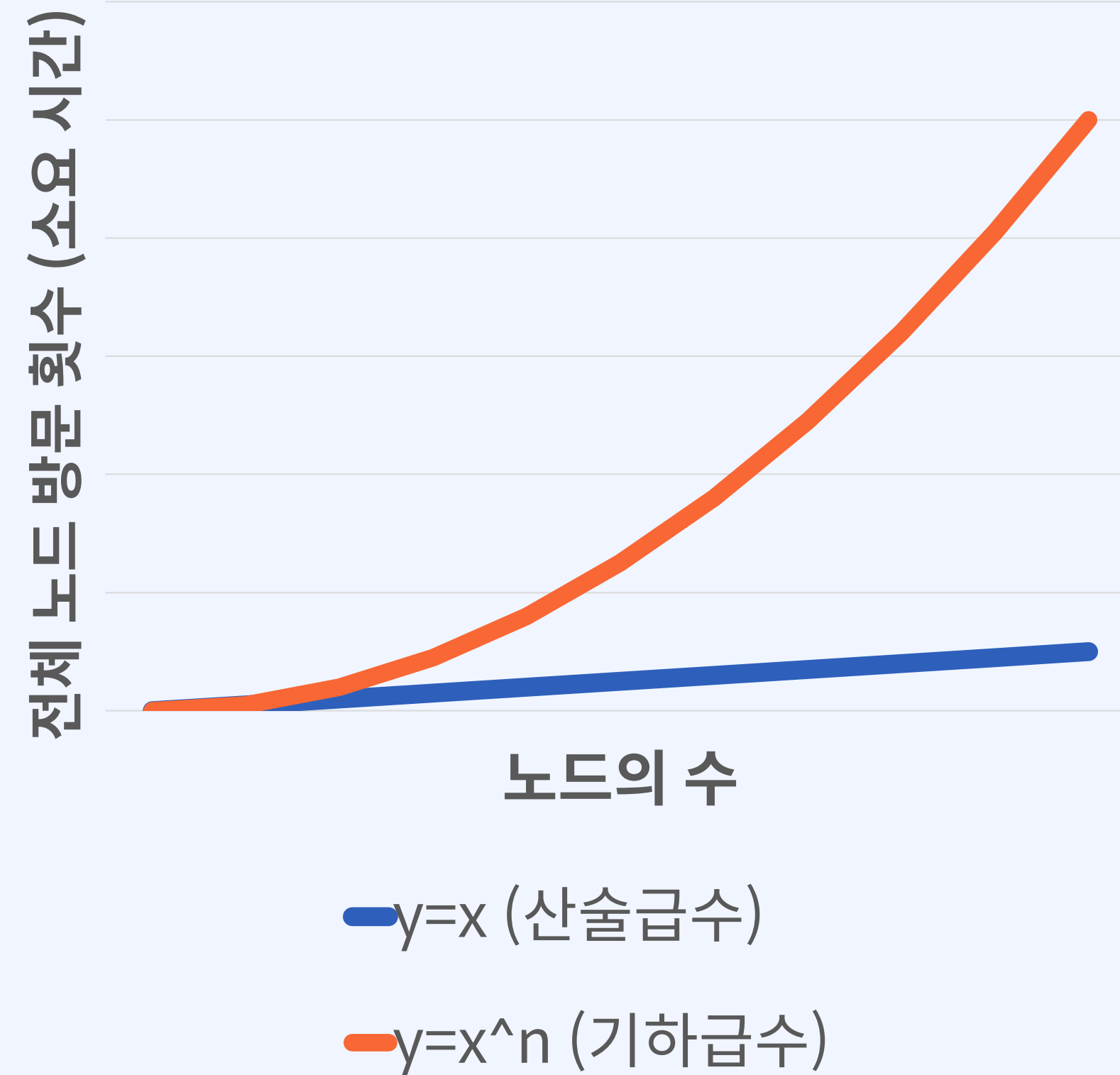
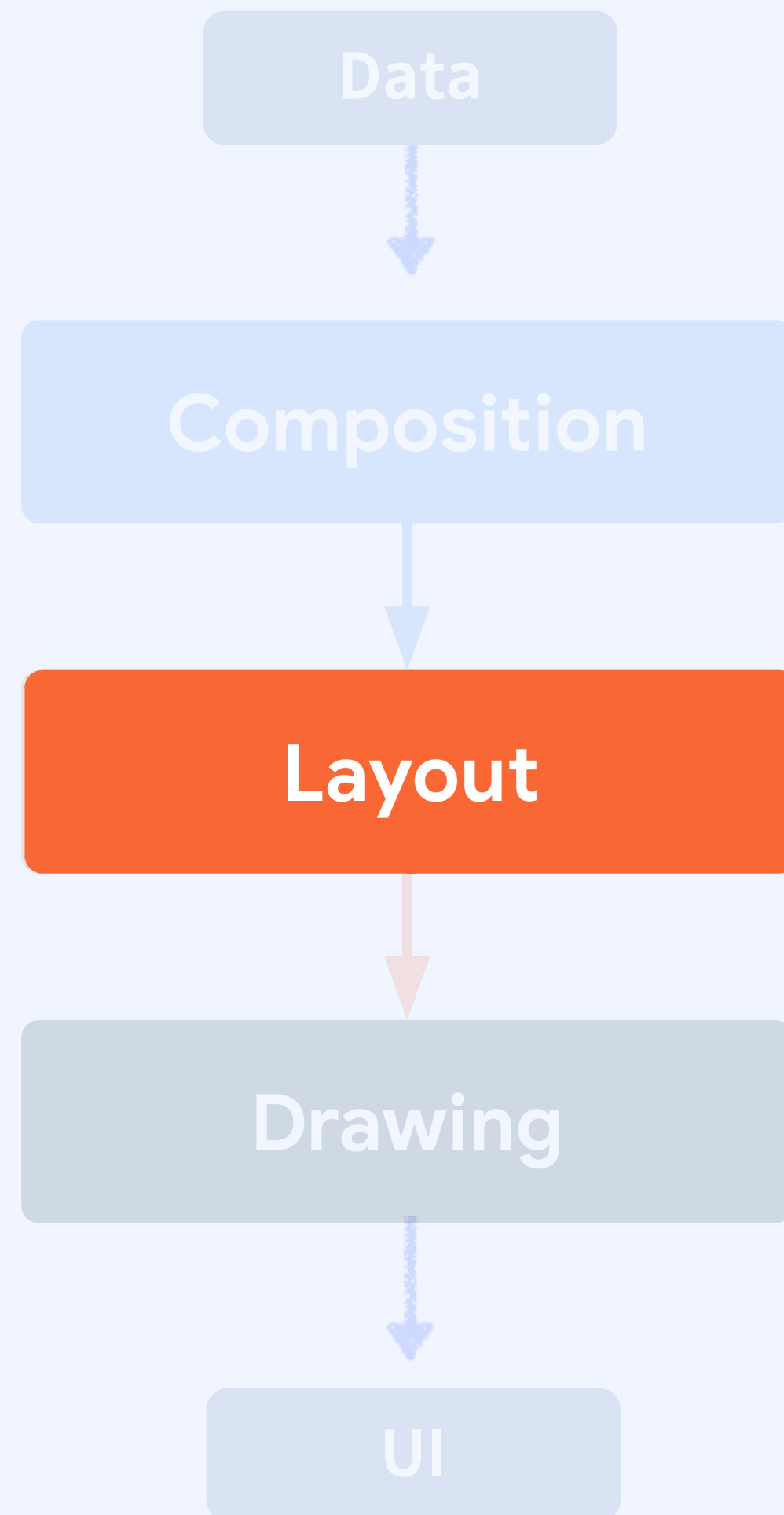
3. Place children (자식 배치)



Rendering Phase Layout

3.

렌더링 단계
Rendering Phase

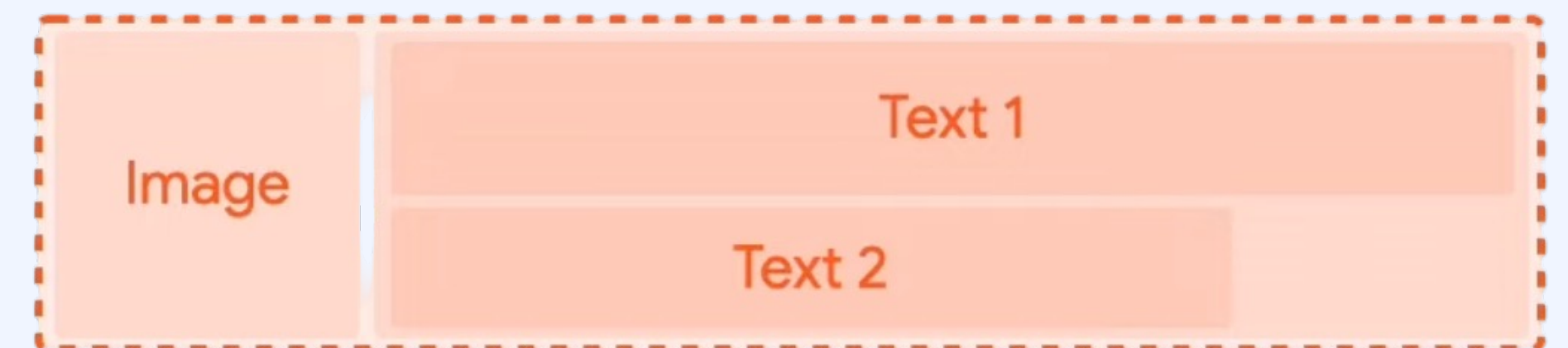
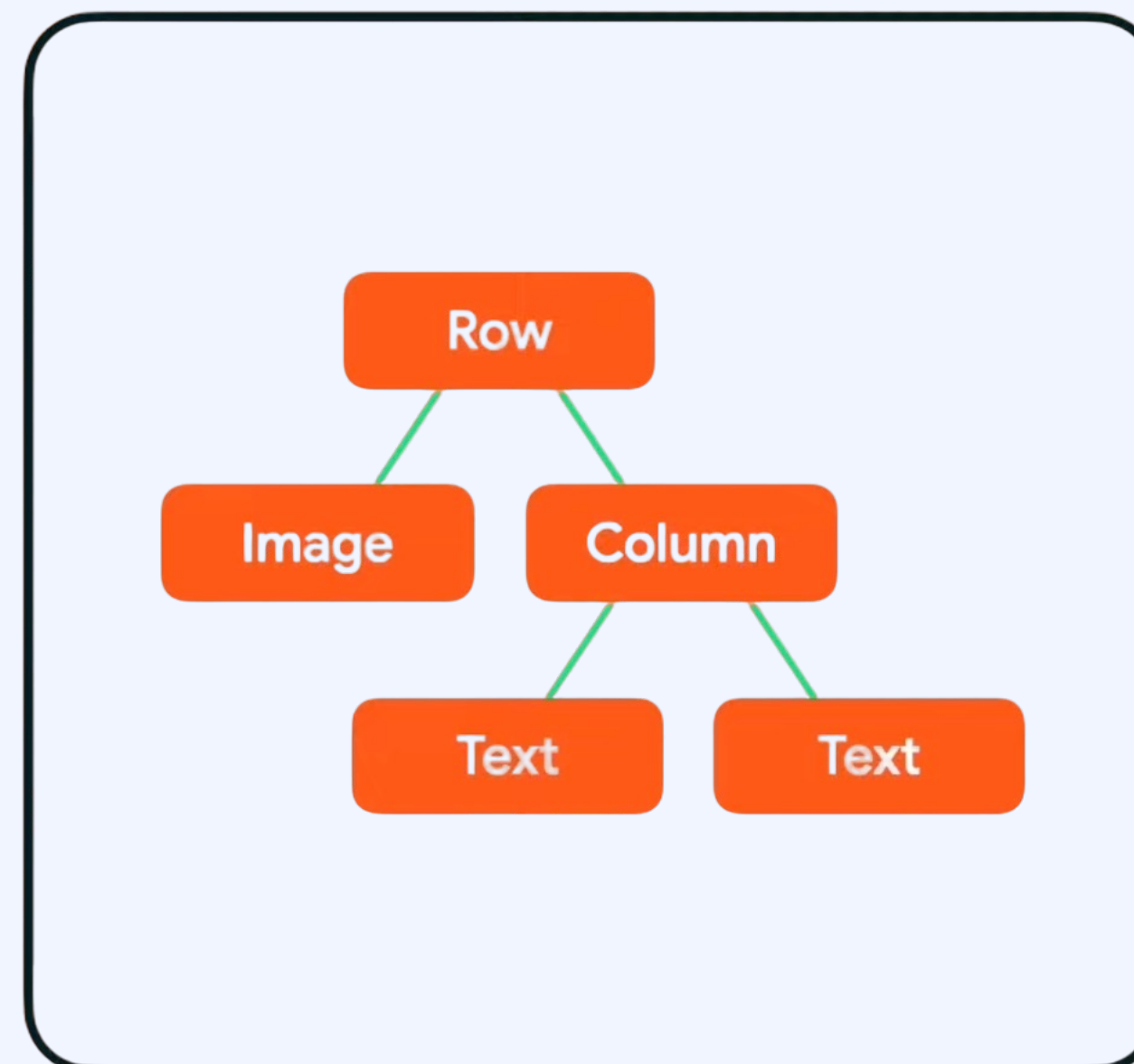
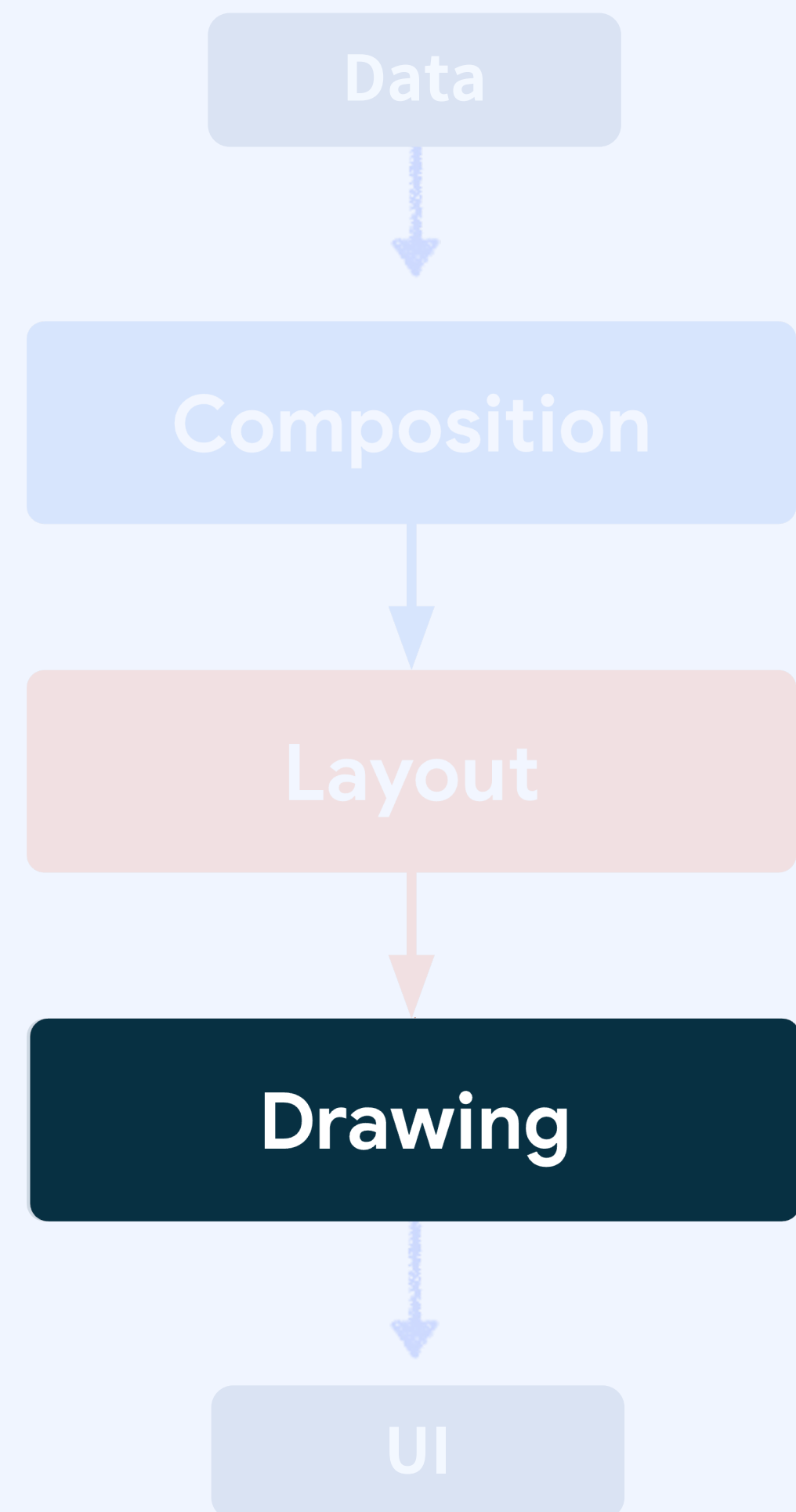


모든 노드를 오직 한번 방문 = 산술급수적 소요시간

Rendering Phase Drawing

3.

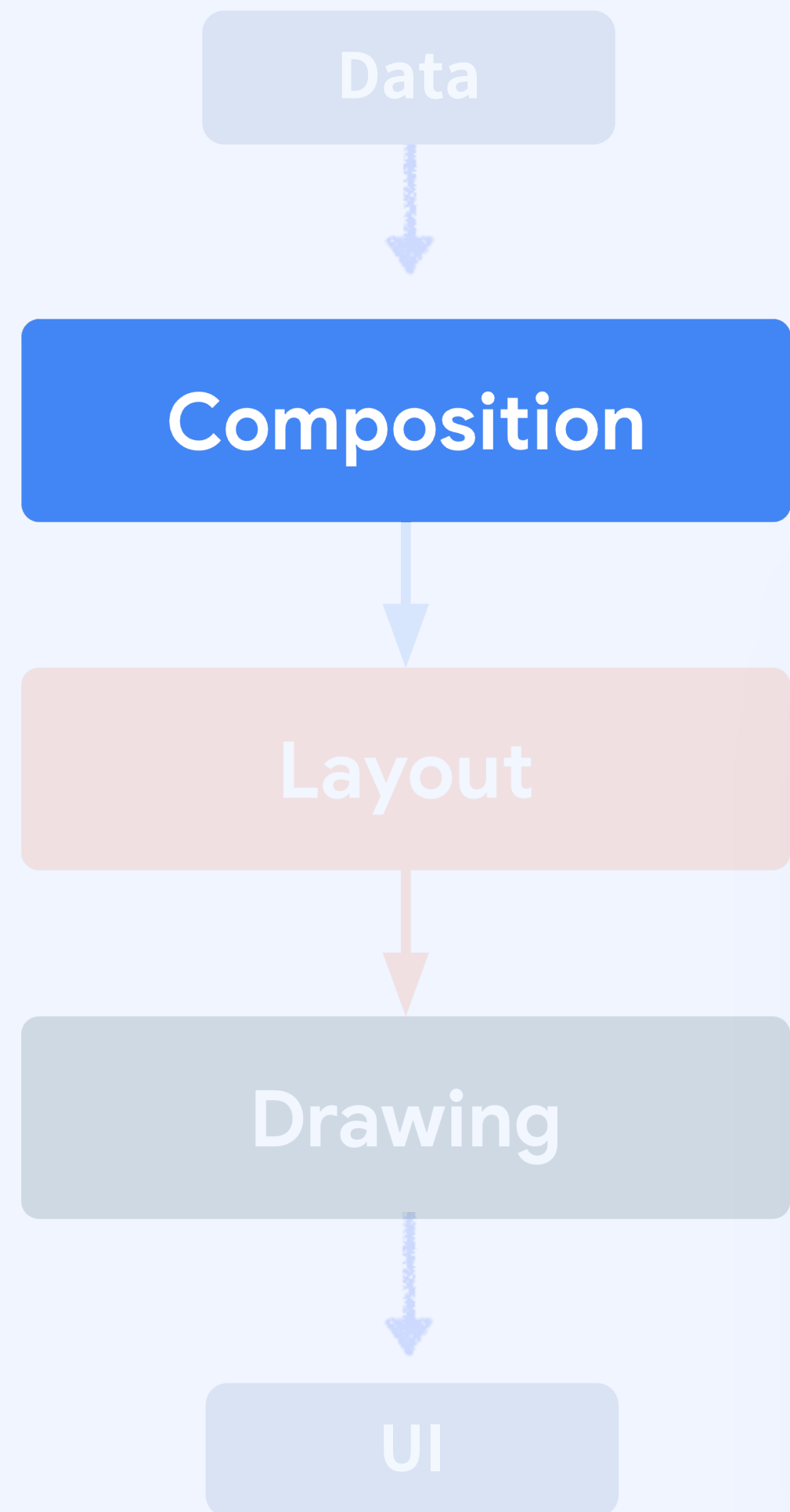
렌더링 단계
Rendering Phase



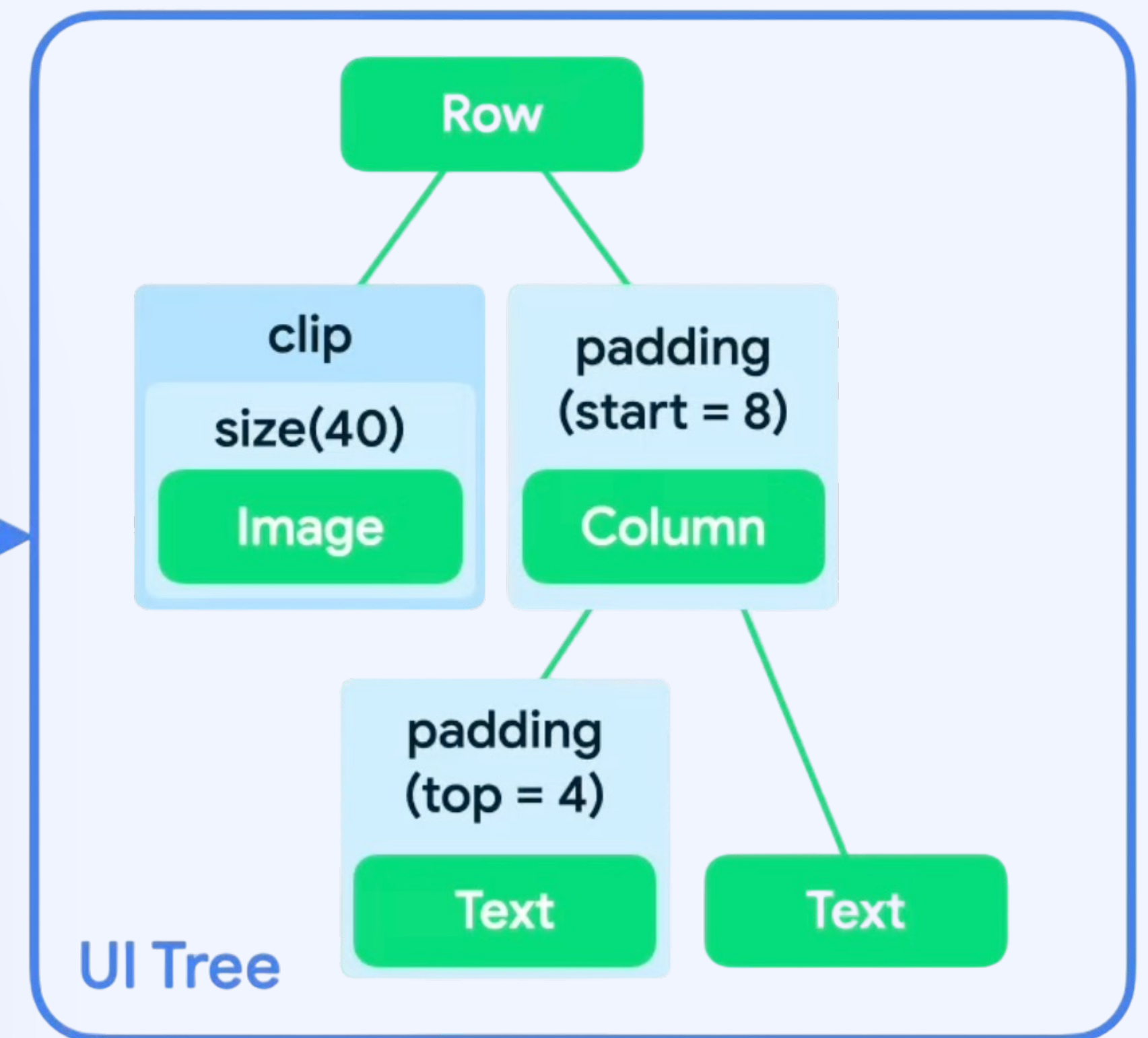
Rendering Phase

Composition - Modifiers

3.
렌더링 단계
Rendering Phase



```
Row {  
  Image(  
    Modifier  
      .clip(CircleShape)  
      .size(40.dp)  
  )  
  Column(  
    Modifier.padding(start = 8.dp)  
  ) {  
    Text(  
      Modifier.padding(top = 4.dp)  
    )  
    Text(...)  
  }  
}
```



Compose 성능 최적화

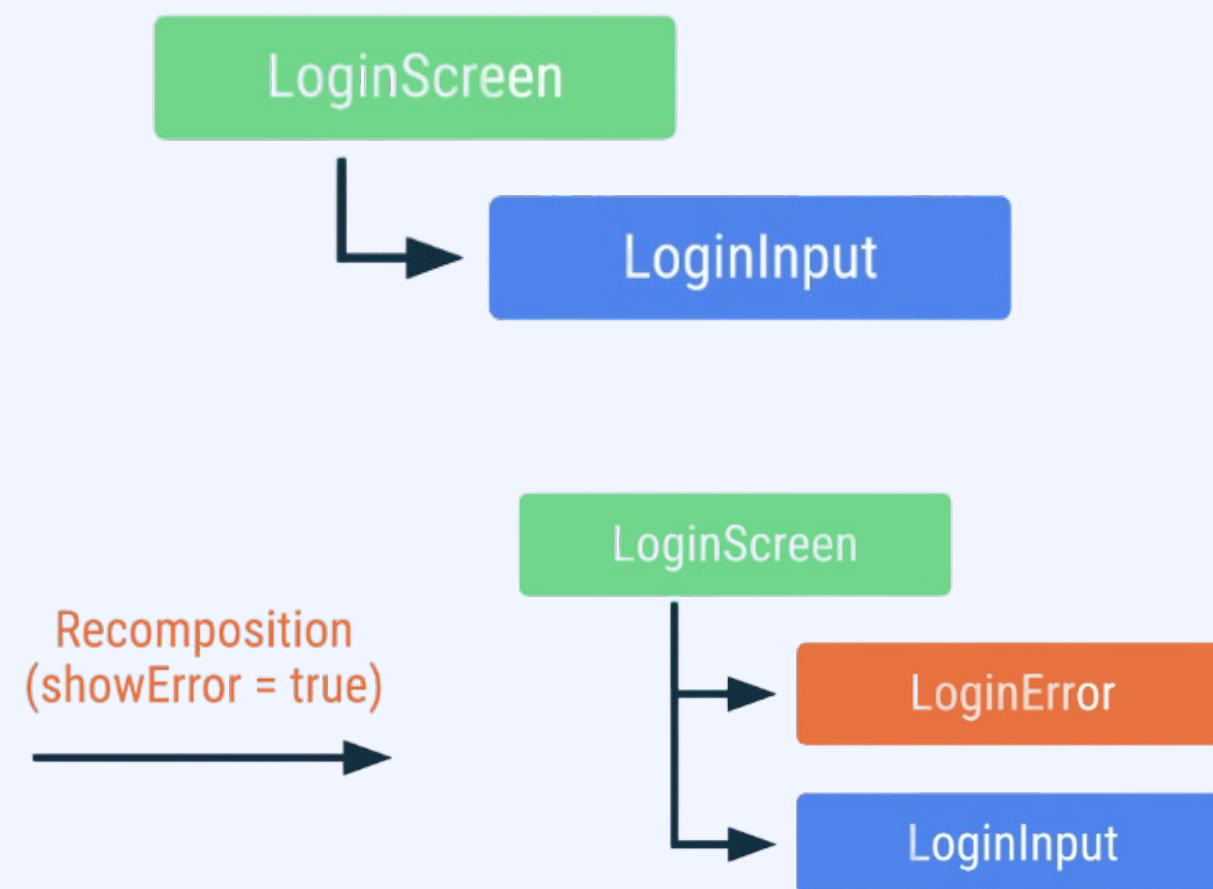
4 안정성 (Stability)

Stability

Smart Recomposition이 뭐였지?

4.

안정성
Stability



Smart Recomposition?

성능 향상을 위해 변경된 부분만 다시 그리기

그렇다면?

컴포저블 함수의 콜사이트나 실행순서가 같다면,
(즉 같은 컴포넌트라면)

리컴포지션이 필요한지 아닌지 Compose가 어떻게 알 수 있을까?

```

@Composable
fun LoginScreen(showError: Boolean) {
    if (showError) {
        LoginError()
    }
    LoginInput()
}

@Composable
fun LoginInput() { /* ... */ }
  
```


Stability

매개변수의 3가지 안정성 유형

4.

안정성
Stability

컴포저블 함수
매개변수의
클래스 타입

Unstable

데이터 변경 가능
변경시 컴포지션에서 추적 불가

Stable

데이터 변경 가능
변경시 컴포지션에서 추적 가능

Immutable

데이터 변경 불가
안정적인 데이터로 취급
원시타입 (String, Int, Float)

Immutable

4.

안정성
Stability

```
fun ContactRow(  
    contact: Contact,  
    modifier: Modifier = Modifier  
) {  
    var selected by remember { mutableStateOf(false) }  
    Row(modifier) {  
        ContactDetails(contact)  
        ToggleButton(  
            selected,  
            onToggled = { selected = !selected }  
        )  
    }  
}
```

Contact

```
data class Contact(  
    val name: String,  
    val number: String  
)
```

ContactDetails

ToggleButton

Mutable

4.

안정성
Stability

```
fun ContactRow(  
    contact: Contact,  
    modifier: Modifier = Modifier  
) {  
    var selected by remember { mutableStateOf(false) }  
    Row(modifier) {  
        ContactDetails(contact)  
        ToggleButton(  
            selected,  
            onToggled = { selected = !selected }  
        )  
    }  
}
```

Contact

```
data class Contact(  
    var name: String,  
    var number: String  
)
```

ContactDetails

ToggleButton

컴파일러에서 보는 컴포즈 컴파일러 리포트 출력

4.

안정성
Stability

```
.gradlew -Pandroidx.enableComposeCompilerReports=true :compose:runtime:runtime:compileKotlin
```

Release 빌드에서 실행해야 정확한 결과값을 얻을 수 있음.

<modulename>-composables.txt / csv

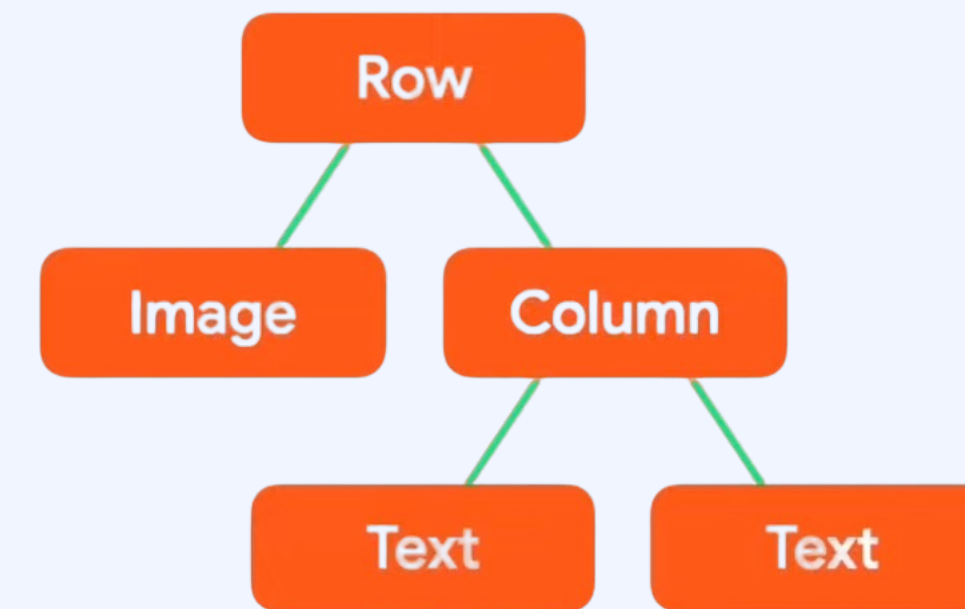
<modulename>-classes.txt

컴파일러에서 보는 컴포즈 skippable restartable

4.
안정성
Stability

```
restartable skippable scheme("[androidx.compose.ui.UiComposable]")  
fun SnackCollection(  
    stable snackCollection: SnackCollection  
    stable onSnackClick: Function1<Long, Unit>  
    stable modifier: Modifier? = @static Companion  
    stable index: Int = @static 0  
    stable highlight: Boolean = @static true  
)
```

<modulename>-composables.txt



skippable

restartable

컴파일러에서 보는 컴포즈 stable / unstable

4.

안정성
Stability

```
restartable scheme("[androidx.compose.ui.UiComposable]") fun  
HighlightedSnacks(  
    stable index: Int  
    unstable snacks: List<Snack>  
    stable onSnackClick: Function1<Long, Unit>  
    stable modifier: Modifier? = @static Companion  
)
```

<modulename>-composables.txt

stable

unstable

컴파일러에서 보는 컴포즈 Collection

4. 안정성 Stability

```
unstable class Snack {
    stable val id: Long
    stable val name: String
    stable val imageUrl: String
    stable val price: Long
    stable val tagline: String
    unstable val tags: Set<String>
    <runtime stability> = Unstable
}
```

<modulename>-classes.txt

```
data class Snack(
    val id: Long,
    val name: String,
    val imageUrl: String,
    val price: Long,
    val tagline: String = "",
    val tags: Set<String> = emptySet()
)
```

그렇다면 왜 Set은 unstable할까?

```
val set: Set<String> = mutableSetOf("foo")
```


Smart Recomposition 전략 Immutable Class

4.

안정성
Stability

```
@Composable
private fun HighlightedSnacks(
    index: Int,
    snacks: ImmutableList<Snack>,
    onSnackClick: (Long) -> Unit,
    modifier: Modifier = Modifier
)
```

```
restartable skipable scheme("[androidx.compose.ui.UiComposable]")
fun HighlightedSnacks(
    stable index: Int
    stable snacks: ImmutableList<Snack>
    stable onSnackClick: Function1<Long, Unit>
    stable modifier: Modifier? = @static Companion
)
```


Smart Recomposition 전략 어노테이션 (Annotations)

4.
안정성
Stability

```
@Immutable
data class SnackCollection(
    val snacks: List<Snack>
)

@Composable
private fun HighlightedSnacks(
    index: Int,
    snacks: SnackCollection,
    onSnackClick: (Long) -> Unit,
    modifier: Modifier = Modifier
)
```

@Immutable

@Stable

Compose 성능 최적화

5 실습 - 상태 읽기 연기 (Defer State Reading)

메모 앱 리팩토링

Recomposition이 계속되는
안티패턴 어플리케이션

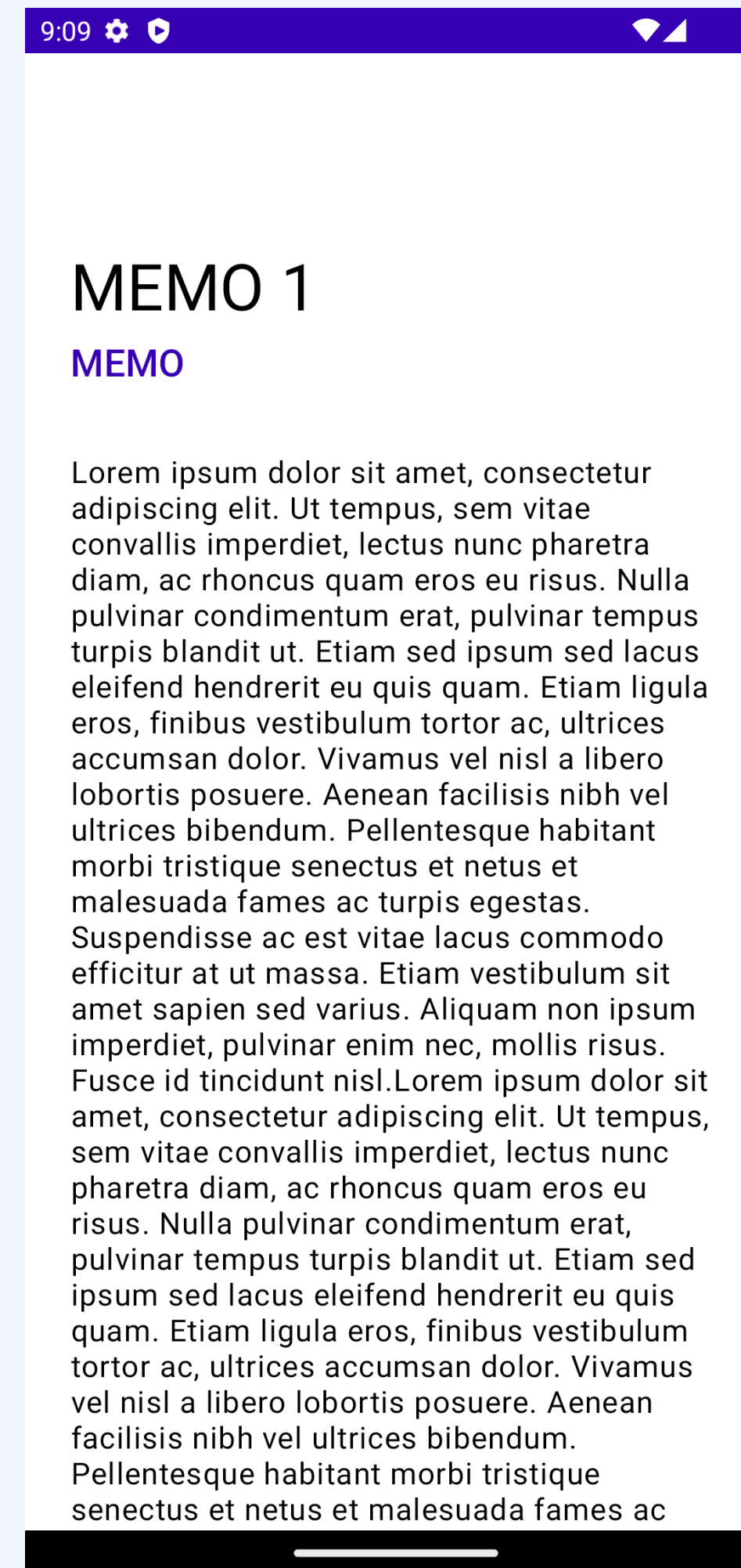
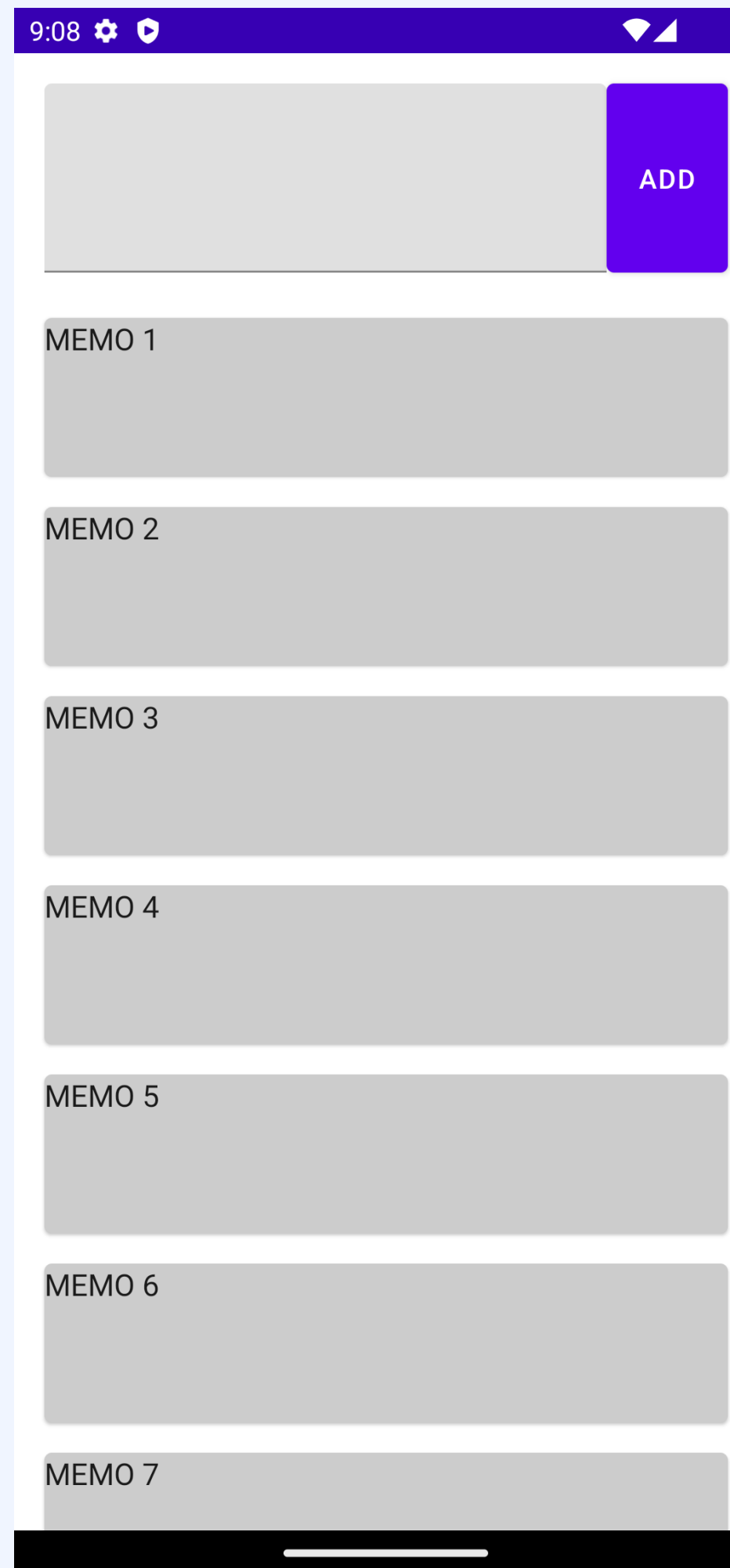
Defer State Reading
(상태 읽기 연기)

Lazy Layout Key
(레이지 레이아웃 키 사용)

Minimize Calculation
(계산 최소화)

Avoid Backward Writes
(역방향 쓰기 방지)

Refactoring



5.

상태 읽기 연기
Defer State
Reading

상태 읽기 연기

5.

상태 읽기 연기
Defer State
Reading

